Heaps

Chapter 17

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

- A heap is a complete binary tree that either is
  - Empty or …
  - Whose root contains a value ≥ each of its children and has heaps as its subtrees
- It is a special binary tree … different in that
  - It is ordered in a weaker sense
  - it will always be a complete binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

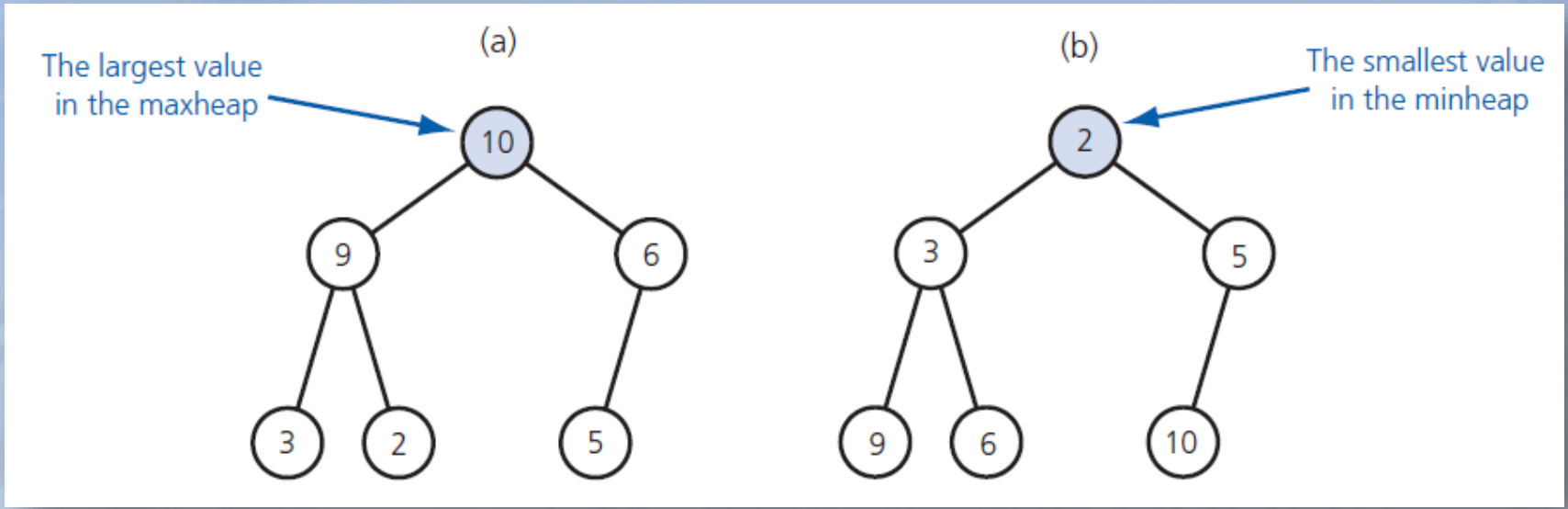


FIGURE 17-1 (a) A maxheap and (b) a minheap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

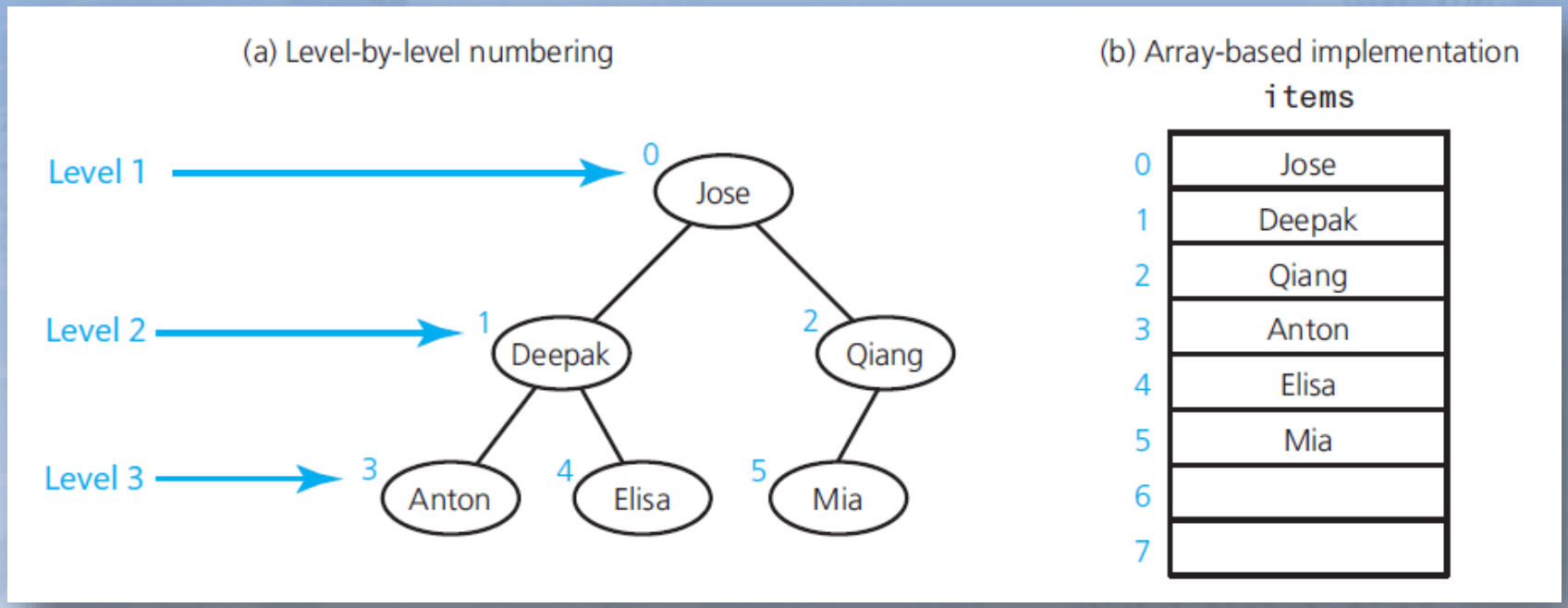
The ADT Heap

```
Heap
---
---
+isEmpty(): boolean
+getNumberOfNodes(): integer
+getHeight(): integer
+peekTop(): ItemType
+add(newData: ItemType): boolean
+remove(): boolean
+clear(): void
```

FIGURE 17-2 UML diagram for the class Heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Array-Based Implementation of a Heap



Given any node items[i], you can easily locate both of its children and its parent:

- Its left child, if it exists, is items[2 * i + 1]
- Its right child, if it exists, is items[2 * i + 2]
- Its parent, if it exists, is items[(i - 1) / 2]

Figure 17-3 A complete binary tree and its array-based implementation

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations

- Assume following private data members
  - items: an array of heap items
  - itemCount: an integer equal to the number of items in the heap
  - maxItems: an integer equal to the maximum capacity of the heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations

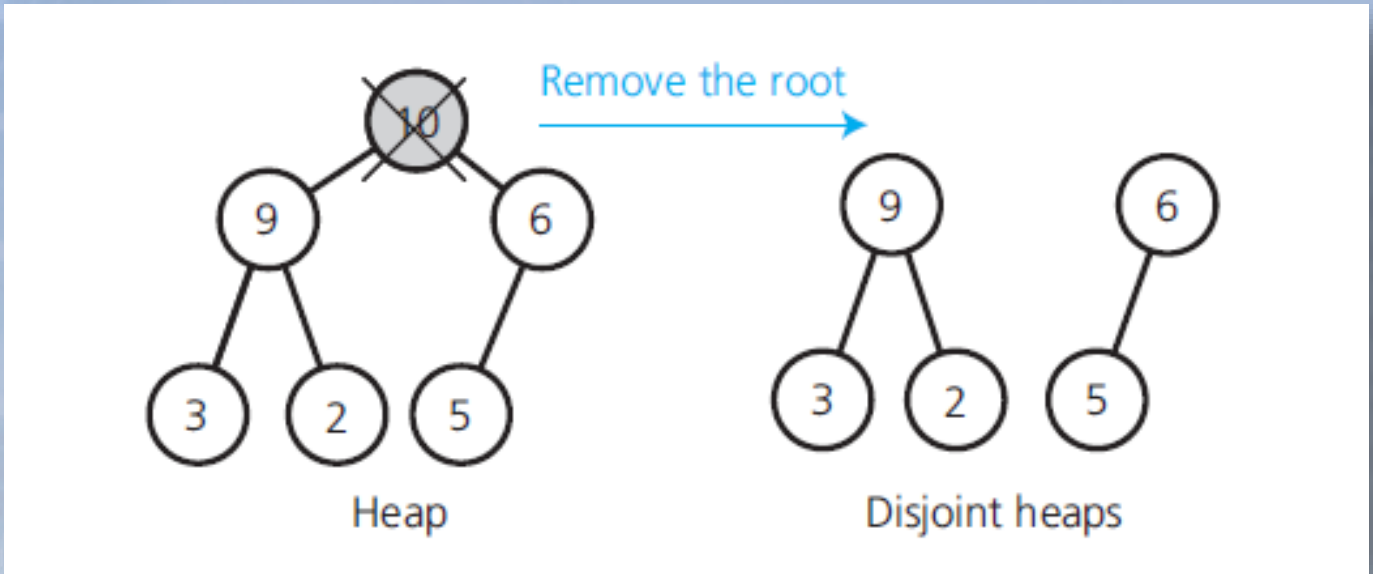
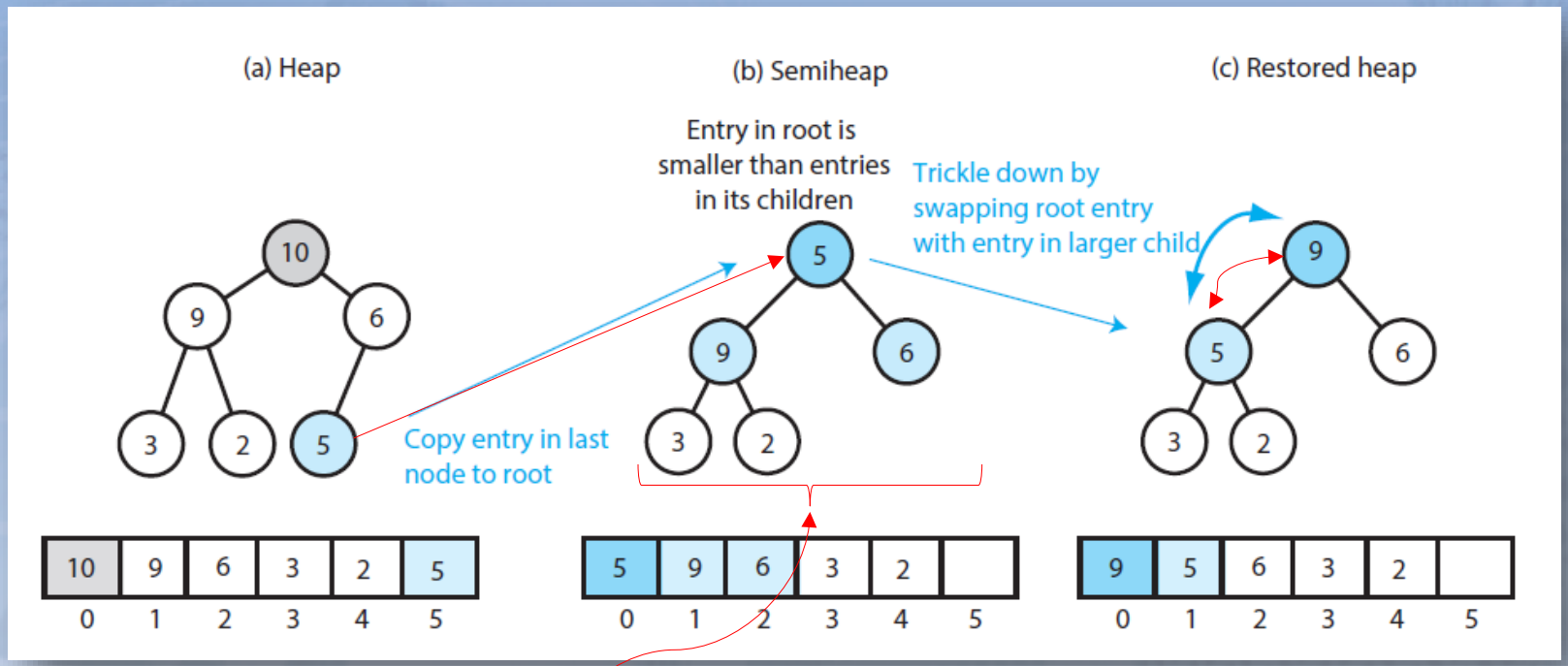


FIGURE 17-4 Disjoint heaps after removing the heap's root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations



Remove root item (10) from a heap.

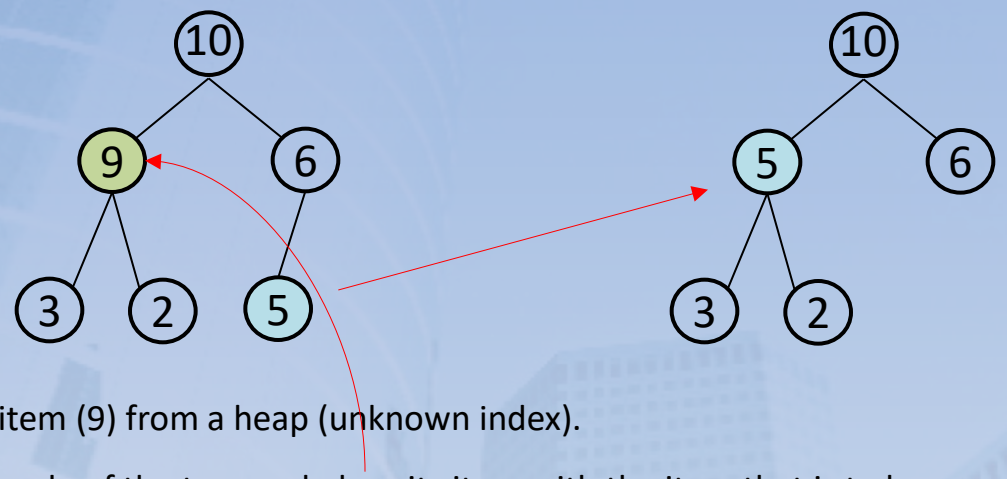
1. remove the last node of the tree and place its item in the root
2. If it is not a heap, then it is a semiheap. Recursively, trickle down the item in the root until it reaches a node in which it will not be out of place (heapify)

Removal efficiency.

- This item travels down a single path from the root to, at worst, a leaf.
- The number of array items it must swap is no greater than the height of the tree.
- Thus, remove is O(log n), which is in fact quite efficient.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations



Remove an arbitrary item (9) from a heap (unknown index).

1. remove the last node of the tree and place its item with the item that is to be removed
2. If it is not a heap, then it is a semiheap. Recursively, trickle down the item in the root until it reaches a node in which it will not be out of place (heapify)

Removal efficiency.

- Find the arbitrary node which is O(n)
- Swap with the last node which is O(1)
- Remove the last node which is O(1)
- Heapify (up or down) which is O(log n)
- Thus, to remove an arbitrary node is O(n + log n), which is O(n).

Algorithms for Array-Based Heap Operations

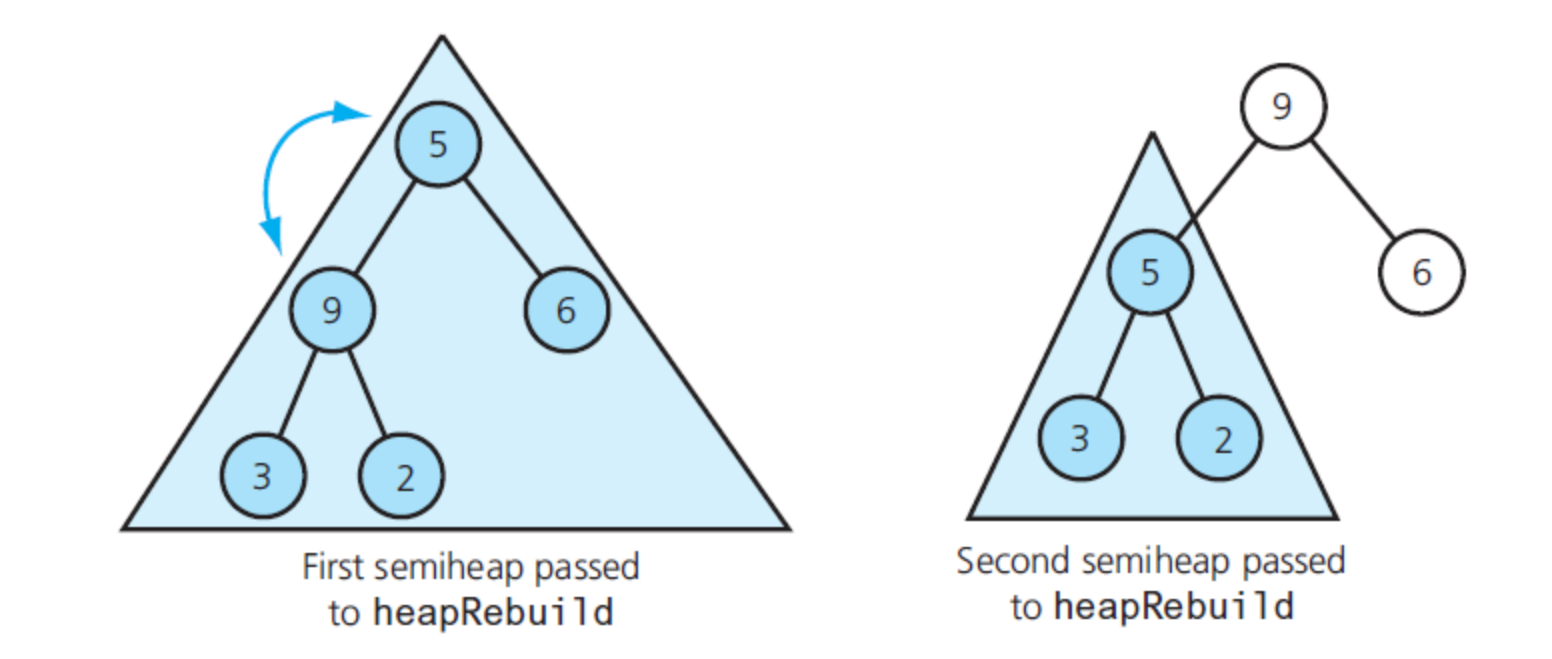
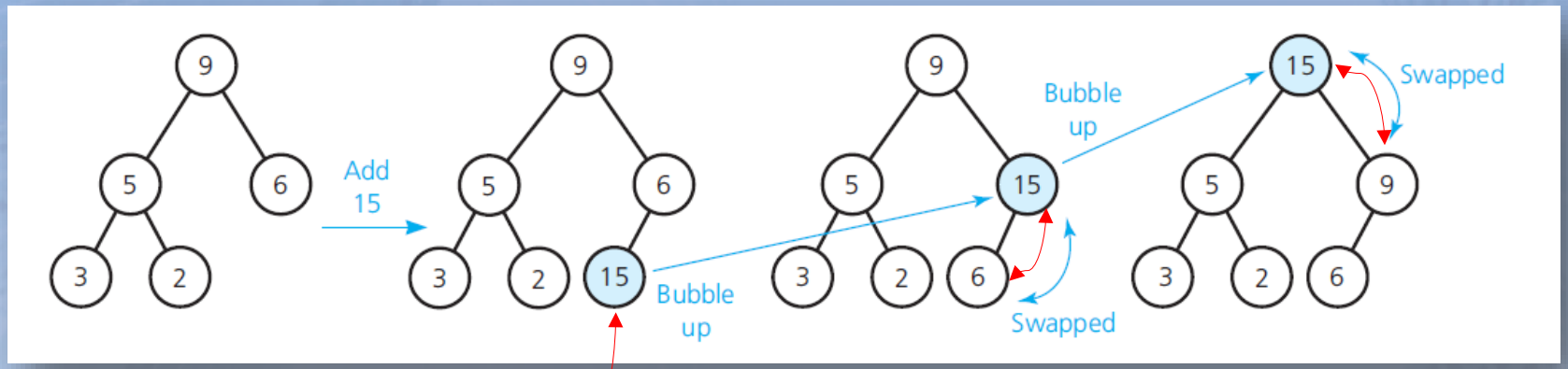


FIGURE 17-6 Recursive calls to heapRebuild

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations



Add an item (15) to a heap (opposite of remove).

1. Insert new item at the bottom of the tree
2. Recursively trickle up to its proper place (the parent of the node in items[i] is always stored in items[(i - 1) / 2] —unless, of course, the node is the root).

Add efficiency.

- This item travels down a single path from a leaf to the root, at worst.
- The number of array items it must swap is no greater than the height of the tree.
- Thus, add is O(log n), which is in fact quite efficient.

FIGURE 17-5 Adding 15 to a heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Transforming Binary Tree to Heap

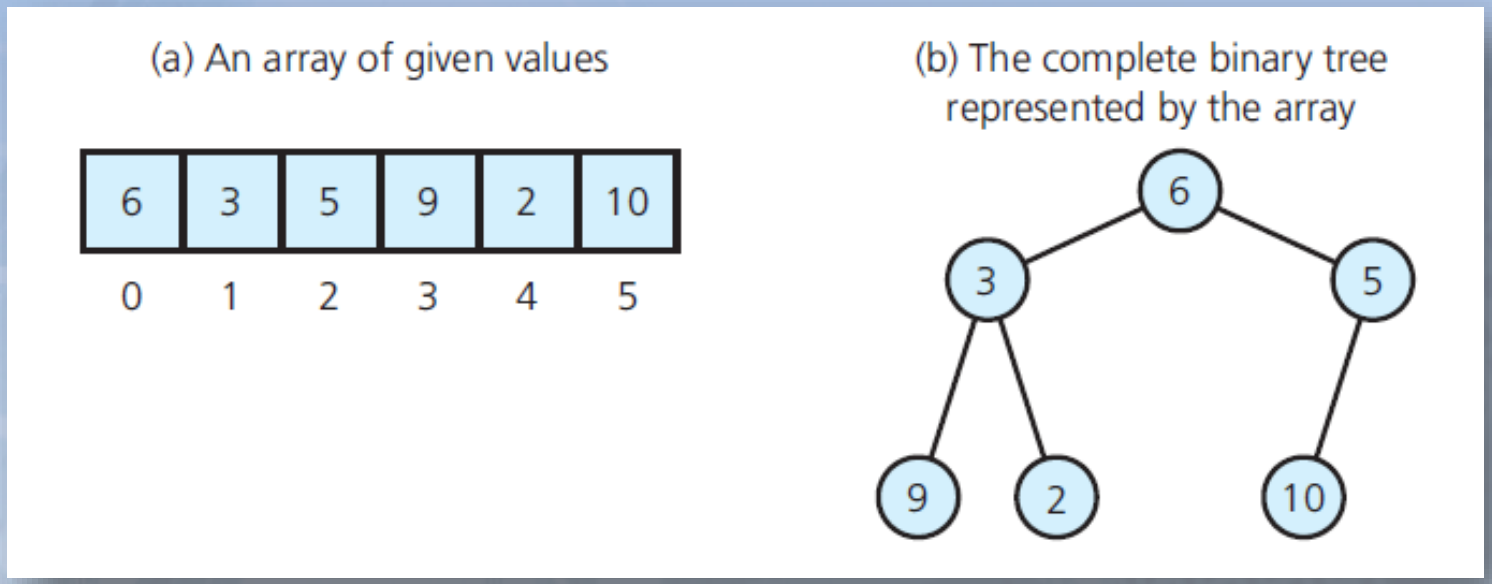
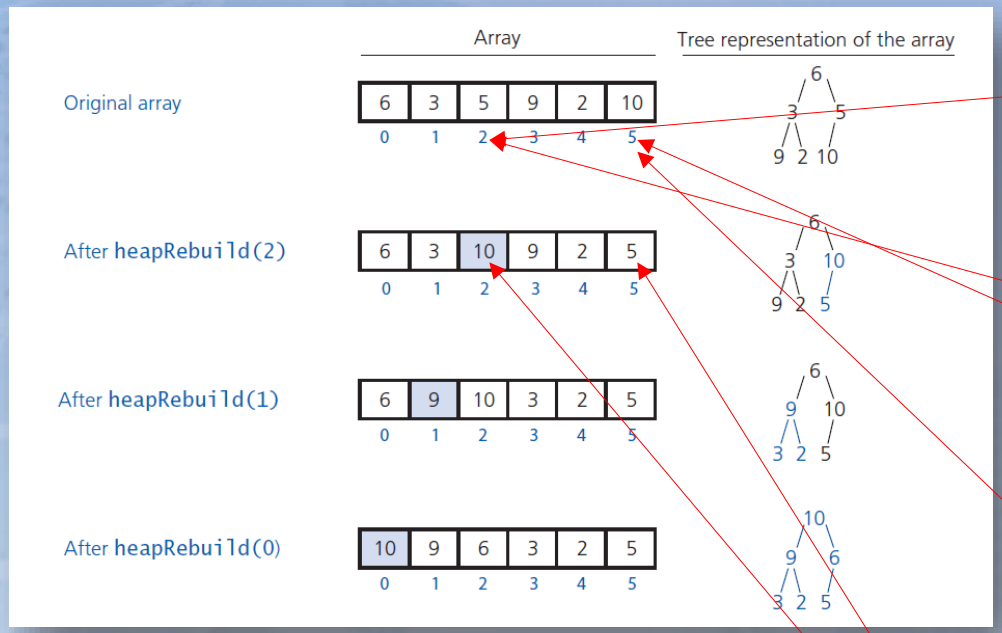


FIGURE 17-8 Array and its corresponding complete binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Transforming Binary Tree to Heap

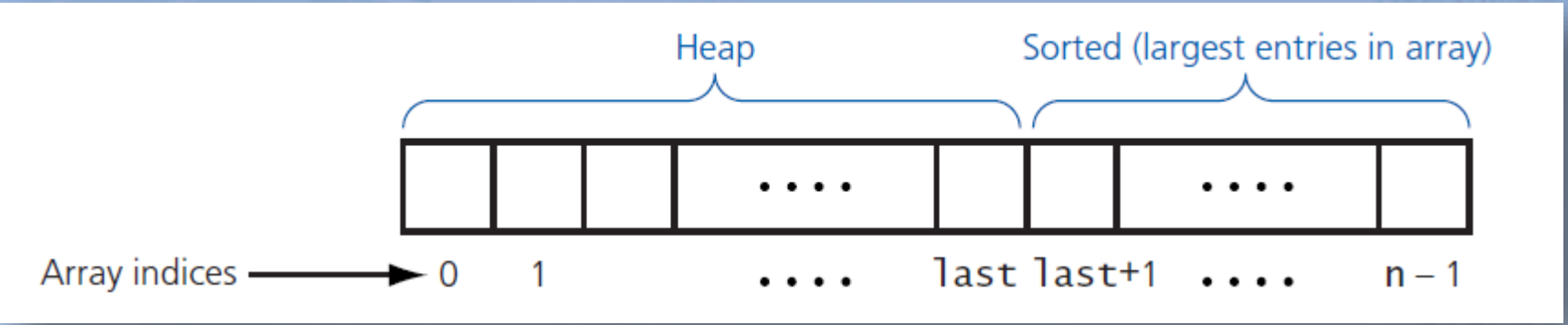



```
function buildMaxHeap(array):
    n = length of array

    # Step 1: Start from the last non-leaf node and move upwards
    for i from n/2 - 1 down to 0:  # from index 2, 1, 0
        heapRebuild(array, n, i)

function heapRebuild(array, n, i):
    largest = i    # initialize largest to root
    leftChild = 2 * i + 1  # index 5, array[leftChild] = 10
    rightChild = 2 * i + 2  # none

    # Step 2: Check if the left child exists and is greater than the
current largest
    if leftChild < n and array[leftChild] > array[largest]:
        largest = leftChild

    # Step 3: Check if the right child exists and is greater than the
current largest
    if rightChild < n and array[rightChild] > array[largest]:
        largest = rightChild

    # Step 4: If largest is not the current node, swap and
heapRebuild the affected subtree
    if largest != i:
        swap(array[i], array[largest])
        heapRebuild(array, n, largest)
```

FIGURE 17-9 Transforming an array into a heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort



heap sort algorithm

1. Transform array into heap
2. For each item from heap
   a. Swap the root (largest item) with last item in the heap
      Swap(myArray[0], myArray[last])
   b. Transform heap region to heap
   c. Decrement last /* reduce heap region/increase sorted region */

FIGURE 17-10 Heap sort partitions an array into two regions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort

heap sort algorithm

1. Transform array into heap
2. For each item from heap
   a. Swap the root (largest item) with last item in the heap
      Swap(myArray[0], myArray[last])
   b. Transform heap region to heap
   c. Decrement last /* reduce heap region/increase sorted region */

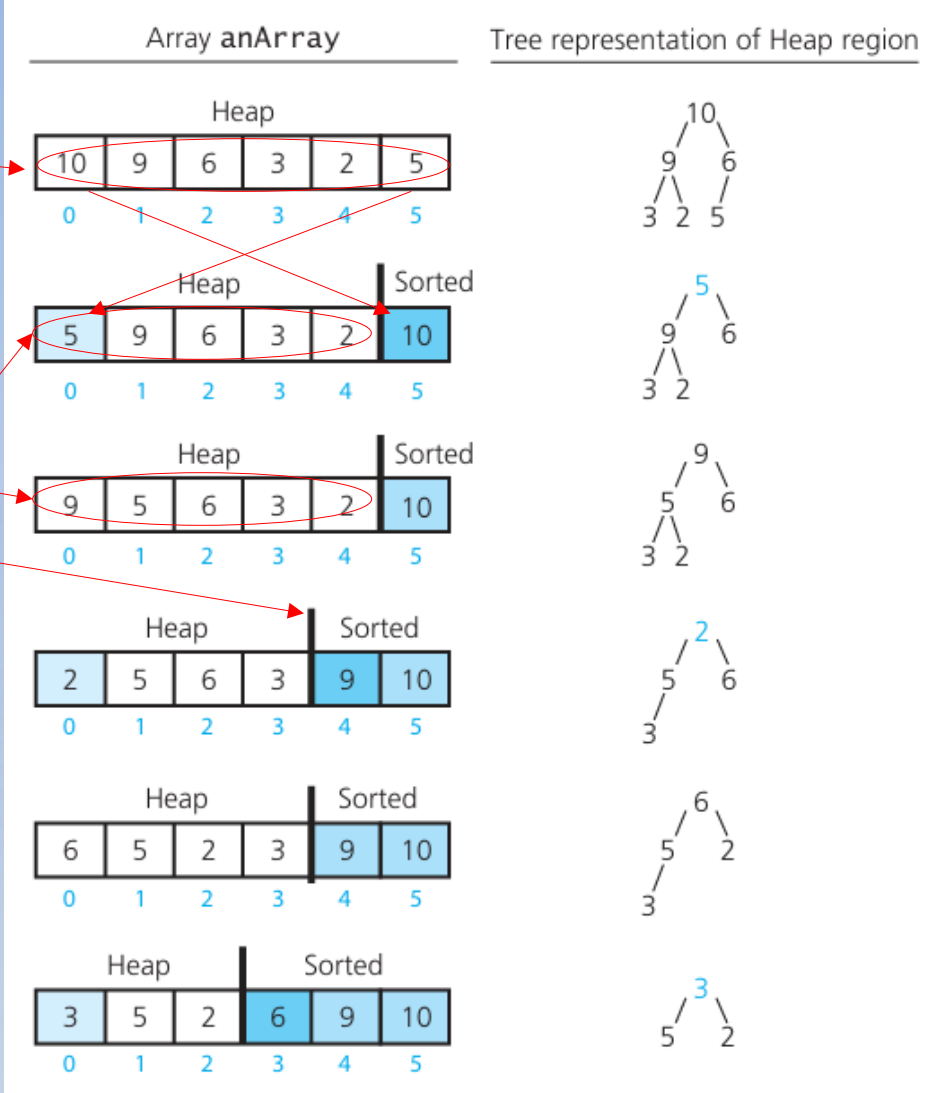


FIGURE 17-11 A trace of heap sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort

heap sort algorithm

1. Transform array into heap
2. For each item from heap
   a. Swap the root (largest item) with last item in the heap
      Swap(myArray[0], myArray[last])
   b. Transform heap region to heap
   c. Decrement last /* reduce heap region/increase sorted region */

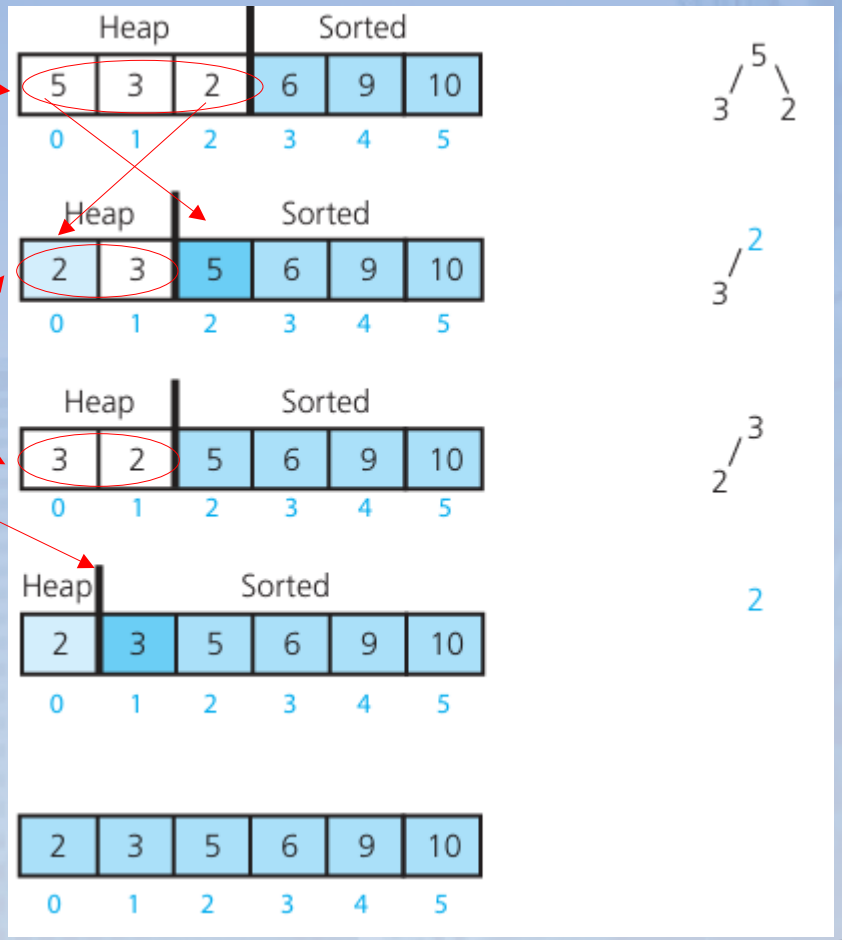


FIGURE 17-11 A trace of heap sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort

- Heap sort converts an array into a heap to locate the array's largest item. This step enables the heap sort to sort an array in an efficient manner.
- Heap sort, like merge sort, has good worst-case and average-case behaviors
  - O(n * log n) // applies to the worst and average cases
- Heap sort has an advantage over merge sort in that it does not require a second array.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Chapter 17

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Chapter 17

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

```
/** Interface for the ADT heap.
 @file HeapInterface.h */

#ifndef HEAP_INTERFACE_
#define HEAP_INTERFACE_

template<class ItemType>
class HeapInterface
{
public:
   /** Sees whether this heap is empty.
    @return  True if the heap is empty, or false if not. */
   virtual bool isEmpty() const = 0;

   /** Gets the number of nodes in this heap.
    @return  The number of nodes in the heap. */
   virtual int getNumberOfNodes() const = 0;
```

LISTING 17-1 An interface for the ADT heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

```
    /** Gets the height of this heap.
     @return  The height of the heap. */
    virtual int getHeight() const = 0;

    /** Gets the data that is in the root (top) of this heap.
        For a maxheap, the data is the largest value in the heap;
        for a minheap, the data is the smallest value in the heap.
     @pre  The heap is not empty.
     @post  The root's data has been returned, and the heap is unchanged.
     @return  The data in the root of the heap. */
    virtual ItemType peekTop() const = 0;

    /** Adds a new data item to this heap.
```

LISTING 17-1 An interface for the ADT heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

```
    /** Adds a new data item to this heap.
     @param newData  The data to be added.
     @post  The heap has a new node that contains newData.
     @return  True if the addition is successful, or false if not. */
    virtual bool add(const ItemType& newData) = 0;

    /** Removes the data that is in the root (top) of this heap.
     @return  True if the removal is successful, or false if not. */
    virtual bool remove() = 0;

    /** Removes all data from this heap. */
    virtual void clear() = 0;

    /** Destroys this heap and frees its assigned memory. */
     virtual ~HeapInterface() {  }
};  // end HeapInterface
#endif
```

LISTING 17-1 An interface for the ADT heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations

```
// Converts a semiheap rooted at index nodeIndex into a heap.
heapRebuild(nodeIndex: integer, items: ArrayType, itemCount: integer): void
{
   // Recursively trickle the item at index nodeIndex down to its proper position by
   // swapping it with its larger child, if the child is larger than the item.
   // If the item is at a leaf, nothing needs to be done.
   if (the root is not a leaf)
   {
      // The root must have a left child; find larger child
      leftChildIndex = 2 * rootIndex + 1
      rightChildIndex = leftChildIndex + 1
      largerChildIndex = rightChildIndex // Assume right child exists and is the larger
```

Recursive algorithm to transform semiheap to heap.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations

```
      // Check whether right child exists; if so, is left child larger?
      // If no right child, left one is larger
      if ((largerChildIndex >= itemCount) ||
                                (items[leftChildIndex] > items[rightChildIndex]))
         largerChildIndex = leftChildIndex; // Assumption was wrong
      if (items[nodeIndex] < items[largerChildIndex])
      {
         Swap items[nodeIndex] and items[largerChildIndex]

         // Transform the semiheap rooted at largerChildIndex into a heap
         heapRebuild(largerChildIndex, items, itemCount)
      }
   }
   // Else root is a leaf, so you are done
}
```

Recursive algorithm to transform semiheap to heap.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations

```
add(newData: itemType): boolean
{
   // Place newData at the bottom of the tree
   items[itemCount] = newData
   // Make new item bubble up to the appropriate spot in the tree
   newDataIndex = itemCount
   inPlace = false
   while ((newDataIndex >= 0) and !inPlace)
   {
      parentIndex = (newDataIndex - 1) / 2
      if (items[newDataIndex] <= items[parentIndex])
         inPlace = true
      else
      {
         Swap items[newDataIndex] and items[parentIndex]
         newDataIndex = parentIndex
      }
   }
   itemCount++
   return inPlace
}
```

Pseudocode for add

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
/** Array-based implementation of the ADT heap.
 @file ArrayMaxHeap.h */
#ifndef ARRAY_MAX_HEAP_
#define ARRAY_MAX_HEAP_

#include "HeapInterface.h"
#include "PrecondViolatedExcept.h"

template<class ItemType>
class ArrayMaxHeap : public HeapInterface<ItemType>
{
private:
   static const int ROOT_INDEX = 0;          // Helps with readability
   static const int DEFAULT_CAPACITY = 21; // Small capacity for testing
   std::unique_ptr<ItemType[]> items;        // Array of heap items
   int itemCount;                            // Current count of heap items
   int maxItems;                             // Maximum capacity of the heap

```

LISTING 17-2 The header file for the class ArrayMaxHeap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
    // ------------------------------------------------------------------
    // Most of the private utility methods use an array index as a parameter
    // and in calculations. This should be safe, even though the array is an
    // implementation detail, since the methods are private.
    // ------------------------------------------------------------------

    // Returns the array index of the left child (if it exists).
    int getLeftChildIndex(const int nodeIndex) const;

    // Returns the array index of the right child (if it exists).
    int getRightChildIndex(int nodeIndex) const;

    // Returns the array index of the parent node.
    int getParentIndex(int nodeIndex) const;

    // Tests whether this node is a leaf.
    bool isLeaf(int nodeIndex) const;

```

LISTING 17-2 The header file for the class ArrayMaxHeap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
    // Converts a semiheap to a heap.
    void heapRebuild(int nodeIndex);

    // Creates a heap from an unordered array.
    void heapCreate();

public:
    ArrayMaxHeap();
    ArrayMaxHeap(const ItemType someArray[], const int arraySize);
    virtual ~ArrayMaxHeap();

    // HeapInterface Public Methods:
    bool isEmpty() const;
    int getNumberOfNodes() const;
    int getHeight() const;
    ItemType peekTop() const throw(PrecondViolatedExcept);
    bool add(const ItemType& newData);
    bool remove();
    void clear();
}; // end ArrayMaxHeap
#include "ArrayMaxHeap.cpp"
#endif
```

LISTING 17-2 The header file for the class ArrayMaxHeap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
template<class ItemType>
int ArrayMaxHeap<ItemType>::getLeftChildIndex(const int nodeIndex) const
{
   return (2 * nodeIndex) + 1;
}  // end getLeftChildIndex
```

Definition of method getLeftChildIndex

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
template<class  ItemType>
ArrayMaxHeap<ItemType>::
ArrayMaxHeap(const ItemType someArray[], const int arraySize):
             itemCount(arraySize), maxItems(2 * arraySize)
{

    // Allocate the array
    items = std::make_unique<ItemType[]>(maxItems);

    // Copy given values into the array
    for (int i = 0; i < itemCount; i++)
       items[i] = someArray[i];

    // Reorganize the array into a heap
    heapCreate();
 }  // end constructor
```

Definition of the constructor

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
for (index = itemCount / 2 down to 0)
{
   // Assertion: The tree rooted at index is a semiheap
   heapRebuild(index)
   // Assertion: The tree rooted at index is a heap
}
```

Building a heap from an array of data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Transforming Binary Tree to Heap

```
template<class ItemType>
void ArrayMaxHeap<ItemType>::heapCreate()
{
   for (int index = itemCount / 2; index >= 0; index--)
      heapRebuild(index);
}  // end heapCreate
```

C++ method heapCreate

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
template<class ItemType>
ItemType ArrayMaxHeap<ItemType>::peekTop() const throw(PrecondViolatedExcept)
{
   if (isEmpty())
      throw PrecondViolatedExcep("Attempted peek into an empty heap.");

   return items[0];
}  // end peekTop
```

C++ method peekTop which tests for an empty heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Implementation of the ADT Priority Queue

```
/** ADT priority queue: Heap-based implementation.
 @file HeapPriorityQueue.h */
#ifndef HEAP_PRIORITY_QUEUE_
#define HEAP_PRIORITY_QUEUE_
#include "ArrayMaxHeap.h"
#include "PriorityQueueInterface.h"

template<class ItemType>
class HeapPriorityQueue : public PriorityQueueInterface<ItemType>,
                          private ArrayMaxHeap<ItemType>
{
public:
   HeapPriorityQueue();
   bool isEmpty() const;
   bool enqueue(const ItemType& newEntry);
   bool dequeue();

   /** @pre  The priority queue is not empty. */
   ItemType peekFront() const throw(PrecondViolatedExcept);
}; // end HeapPriorityQueue

#include "HeapPriorityQueue.cpp"
#endif
```

LISTING 17-3 A header file for the class HeapPriorityQueue

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Implementation of the ADT Priority Queue

```
/** Heap-based implementation of the ADT priority queue.
 @file HeapPriorityQueue.cpp */

#include "HeapPriorityQueue.h"

template<class ItemType>
HeapPriorityQueue<ItemType>::HeapPriorityQueue()
{
   ArrayMaxHeap<ItemType>();
}  // end constructor

template<class ItemType>
bool HeapPriorityQueue<ItemType>::isEmpty() const
{
   return ArrayMaxHeap<ItemType>::isEmpty();
}  // end isEmpty

template<class ItemType>
bool HeapPriorityQueue<ItemType>::enqueue(const ItemType& newEntry)
{
   return ArrayMaxHeap<ItemType>::add(newEntry);
}  // end add
```

LISTING 17-4 An implementation of the class HeapPriorityQueue

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Implementation of the ADT Priority Queue

```
template<class ItemType>
bool HeapPriorityQueue<ItemType>::dequeue()
{
   return ArrayMaxHeap<ItemType>::remove();
}  // end remove

template<class ItemType>
ItemType HeapPriorityQueue<ItemType>::peekFront() const throw(PrecondViolatedExcept)
{
   try
   {
      return ArrayMaxHeap<ItemType>::peekTop();
   }
   catch (PrecondViolatedExcept e)
   {
      throw PrecondViolatedExcept("Attempted peek into an empty priority queue.");
   }  // end try/catch
}  // end peekFront
```

LISTING 17-4 An implementation of the class HeapPriorityQueue

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Implementation of the ADT Priority Queue

- Heap versus a binary search tree
  - If you know maximum number of items in the priority queue, heap is the better implementation.
- Finite, distinct priority values
  - Many items likely have same priority value
  - Place in same order as encountered

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort

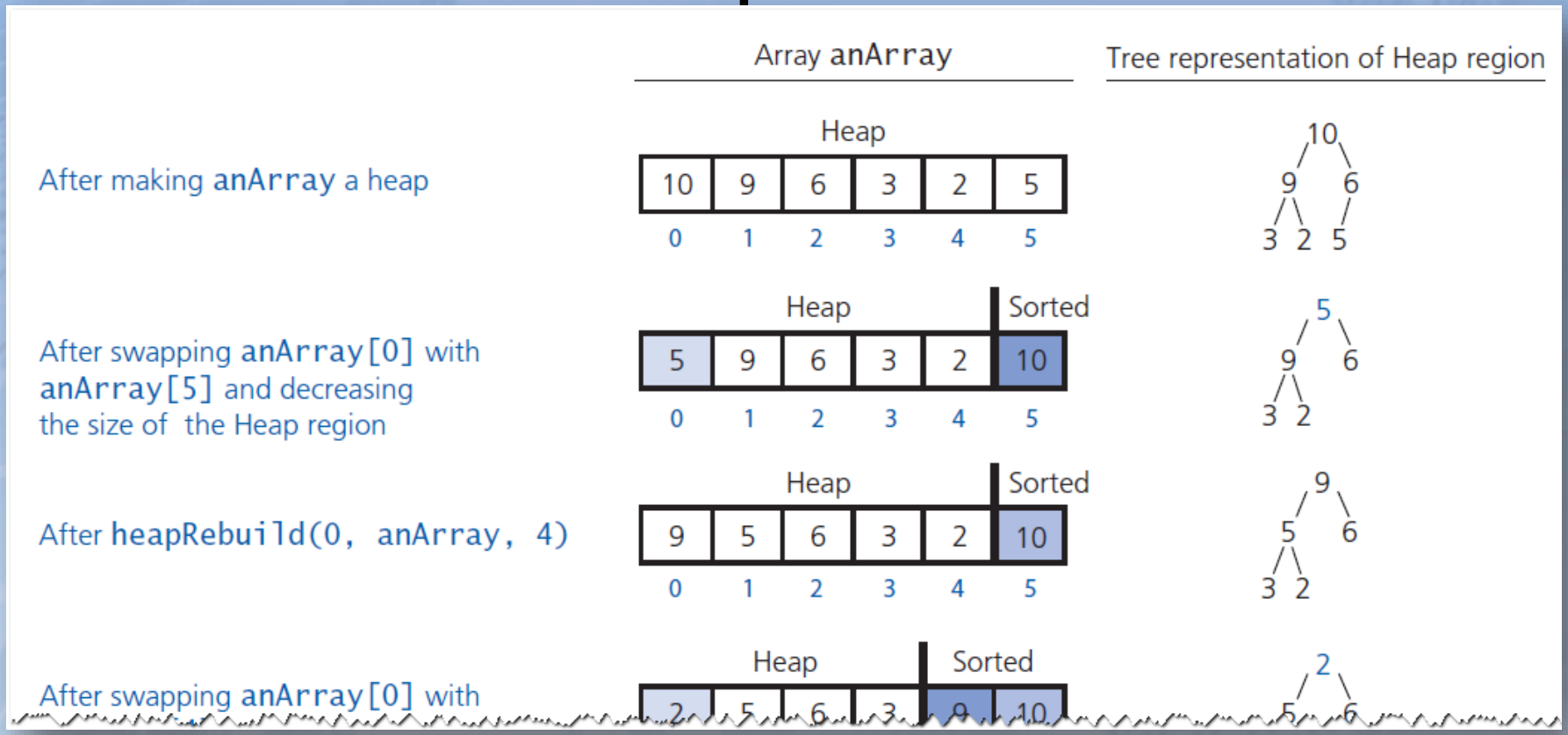


FIGURE 17-11 A trace of heap sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort

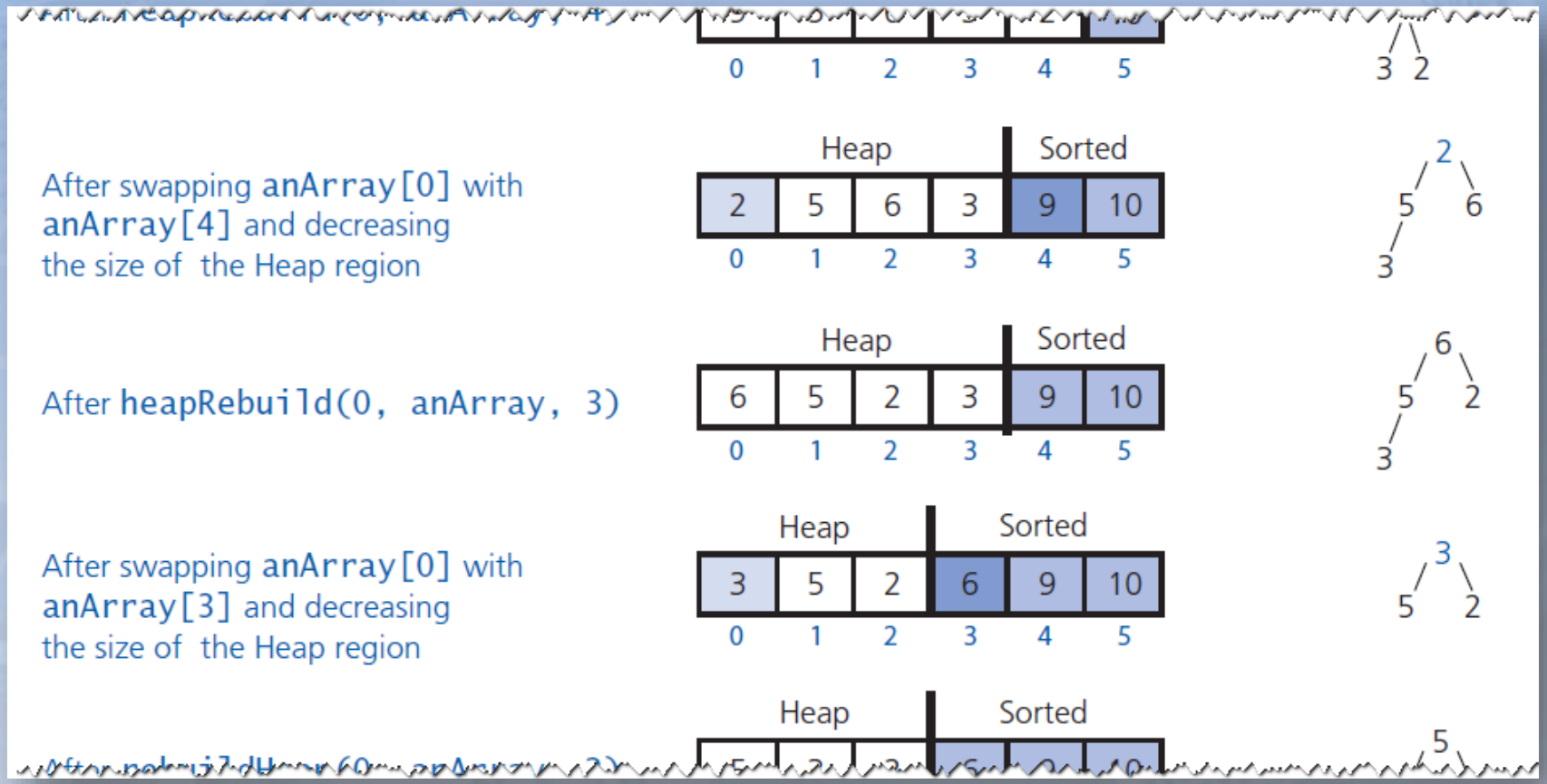


FIGURE 17-11 A trace of heap sort, beginning with the heap in Figure 17-9

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort

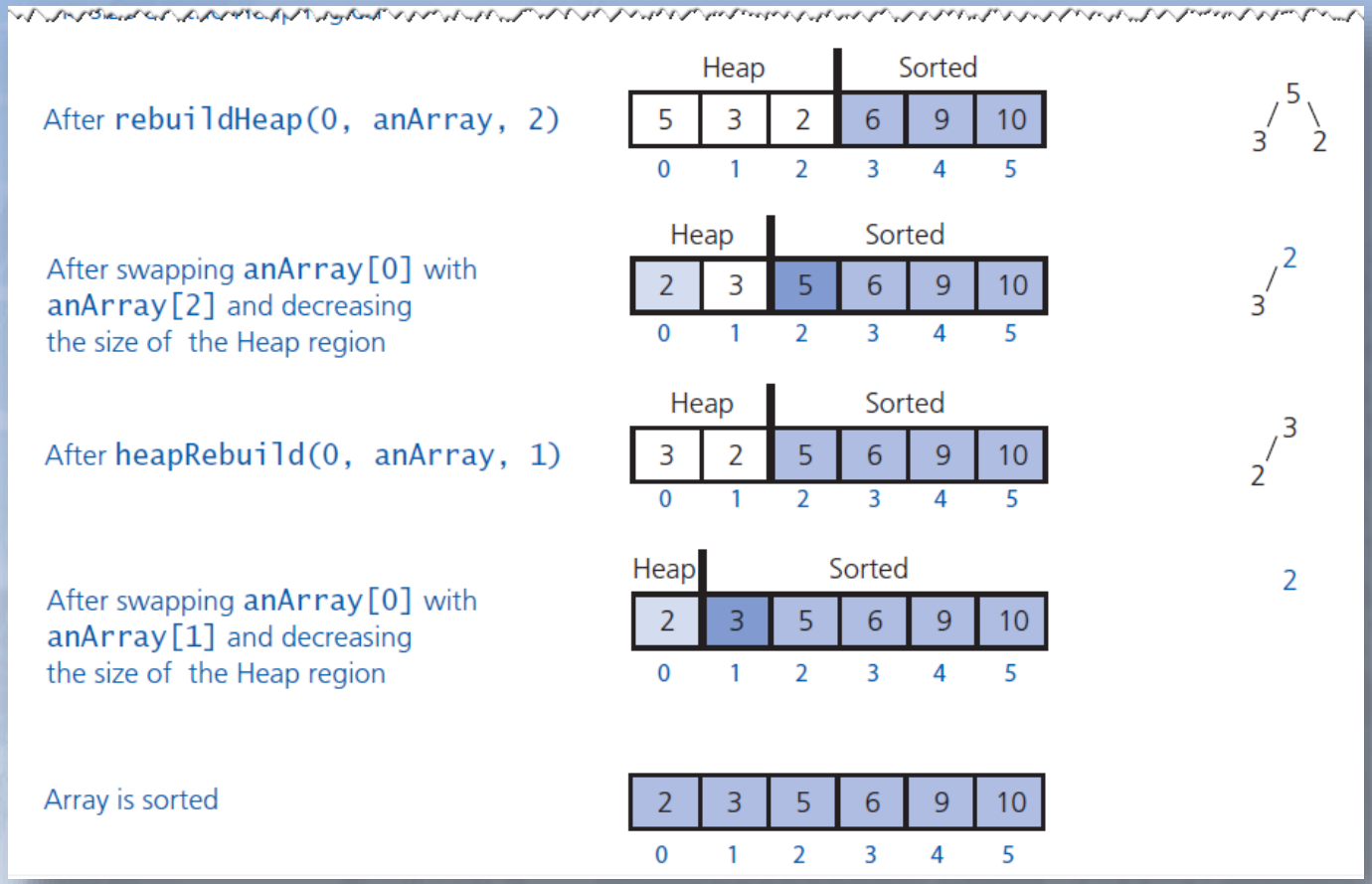


FIGURE 17-11 A trace of heap sort, beginning with the heap in Figure 17-9

© 2017 Pearson Education, Hoboken, NJ. All rights reserved